Autonomous Robotic Fishing Line Tension Regulation Using Admittance Control

Nathan Ge
Dept. of MAE
UCLA
Los Angeles, CA, USA

Chaoyi Hu
Dept. of MAE
UCLA
Los Angeles, CA, USA

Julie Nguyen
Dept. of MAE
UCLA
Los Angeles, CA, USA

Xiang Zhou
Dept. of MAE
UCLA
Los Angeles, CA, USA

***Abstract*—This paper presents an autonomous robotic fishing system focused on post hook-set tension regulation as a force control problem. After a fish is hooked, maintaining line tension requires balancing overpulling, which risks snapping the line, against underpulling, which can lose the fish. Our platform is a one degree-of-freedom robotic rod that measures fishing line tension with a load sensor and uses an admittance controller to convert tension feedback into rod pitch motion. The control architecture combines tension-based regulation with position compensation and an engagement supervisor that manages transitions between idle, active control, and recovery states. We implement the system in a ROS2 software stack and validate it first in MuJoCo simulation with a repeatable fish-like disturbance, then on hardware with manual line tension trials. Simulation and hardware experiments show that the robot responds intelligently to changing line tension rather than applying fixed motor commands. This work demonstrates a practical foundation for sensor-driven tension regulation in robotic fishing and motivates future extensions to multi-DOF systems, improved state switching, and more complete autonomous fishing pipelines.**

***Index Terms*—tension regulation, admittance control, force control, robotic fishing, ROS2**

I. Overview

This project explores the potential of autonomous robotic fishing framework. To carry the problem into an engineering lens to build a technical framework around, we centered our robot's function on the fishing line tension aspect of the fishing pipeline which is modelled as a force control problem. Tension regulation is fundamentally characterized by the balance of overpulling which risks the line snapping and underpulling which can lose the fish. Good anglers will respond by dynamically adjusting the orientation of the rod with intuition and feel, keeping the line tension in a safe zone. The robot will perform this balancing act intelligently with sensor feedback and precise motor control.

Similar robotic fishing systems have been explored in previous projects. AlYaseen et al. developed an autonomous fishing rod that can detect fish bites, pull the line, and stop the motor when high resistance is detected to avoid cutting the line [1]. This is closely related to our project because both systems use the fishing-line force as an important feedback signal. Another robotic fishing pole project used an Arduino-controlled mechanical system to cast and reel automatically through smartphone or computer control [2]. These examples show that fishing is a suitable application for automation and robotic actuation.

Fishing has also been studied as a force-feedback and haptic simulation problem. The Stanford CHARM Lab developed a haptic fishing robot/simulator to create realistic force feedback for the user [3]. This supports the motivation of our project because the key part of the fishing experience is not only moving the rod, but also reacting to the changing resistance from the fish. Our project extends this idea by using a physical robotic rod and an admittance controller to convert measured tension into rod motion.

The resultant robot platform is a 1 DOF robotic fishing system to control line tension following the hook set fishing phase. The robot employs sensor-based feedback control to manage tension regulation and disturbance rejection. The tension in the fishing line is detected by a load sensor, and the robot controls the rod pitch angle based on the error of the detected tension. The control system logic was modeled and solidified in simulation as a tethered robot in which the fish-like load is an external disturbance via the line. The project also creates a foundation for future work on more realistic multi-DOF fishing robots or autonomous fishing mechanisms.

II. Control Objectives

To carry the problem into an engineering lens to build a technical framework around, fishing can be characterized by three discrete modes: fish discovery (pre hook-set), tension regulation (post hook-set), and reeling.

Fish discovery describes the phase in which the fishing line is 'cold' and waiting for a fish to bite. The controller looks for a disturbance signal to indicate fish interest. A jigging action that mimics live bait may be used in this phase to increase fish hooking probability. This phase also includes holes/gaps in tension sensing caused by a fish already hooked on that escapes the jurisdiction of the control domain. Algorithms/action routines may be employed to restore control over the fish.

Looking past fish discovery is the tension regulation stage, the project's primary control objective, which centers on maintaining tension through the fishing line. The general control structure involves a physical load sensor that approximates the force through the line, and control algorithms that take this information to modulate the tension estimation data. Tension

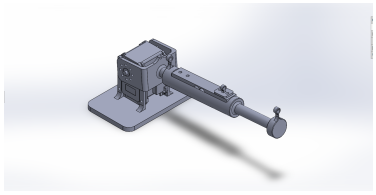
accommodation lends itself naturally to admittance control or some sort of force regulation control methodology, both of which became primary control methods of exploration for this project. Key concepts/considerations include: Tension regulation, Virtual mass-spring-damper (MSD) admittance loops, and Highly stochastic loads handling.

An additional reeling component may be added to successfully bring the fish to the desired endpoint. Reeling will also require additional hardware to pull the string inward. This would complicate the load sensing packaging and add mechanical complexity that comes with a reeling system. This particular system was not heavily pursued in our research, leaving it an avenue of future development.

III. Methods

A. Hardware

(a) Physical robot.



(b) CAD of robot.

Fig. 1: Robot hardware and corresponding CAD assembly.

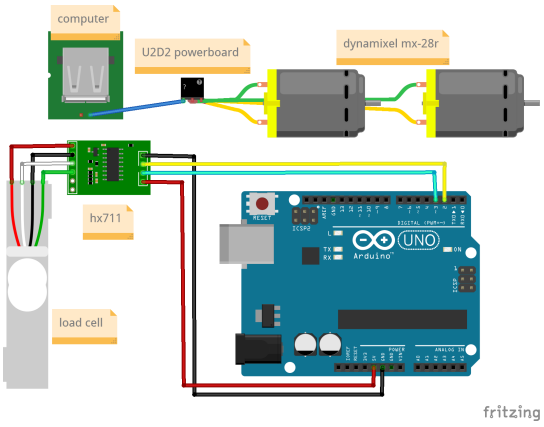


Fig. 2: Wiring diagram.

B. Mathematical Formulation

1) System Definitions: Let the 2-DOF robotic fishing platform be modeled as a spherical manipulator. We define the state variables and physical parameters as follows:

- q_1: The yaw angle of the base around the global Z-axis.
- q_2: The pitch angle of the rod relative to the horizontal XY plane.
- L: The fixed length of the fishing rod from the pitch joint to the tip.
- h: The vertical height offset of the pitch joint from the ground.

2) Forward Kinematics: The forward kinematics map the joint space (q_1, q_2) to the Cartesian task space (x, y, z) of the rod tip. By projecting the rod length onto the horizontal plane, we obtain the radial distance $r = L\cos(q_2)$. Decomposing this radial distance and accounting for the base height yields the position equations:

$$\begin{aligned} x &= L\cos(q_1)\cos(q_2) \\ y &= L\sin(q_1)\cos(q_2) \\ z &= h + L\sin(q_2) \end{aligned}$$

3) Inverse Kinematics: The inverse kinematics determine the required joint angles (q_1, q_2) to reach a desired target coordinate (x, y, z).

The yaw angle q_1 is isolated by taking the ratio of the y and x position equations. To ensure the correct quadrant is selected, the two-argument arctangent function is utilized:

$$q_1 = \text{atan2}(y, x)$$

The pitch angle q_2 is found by relating the vertical displacement of the rod tip to the horizontal radial distance from the base, where $r = \sqrt{x^2 + y^2}$:

$$q_2 = \text{atan2}(z - h, \sqrt{x^2 + y^2})$$

4) Analytical Jacobian: The analytical Jacobian matrix, $\mathbf{J}$, is required for force control and admittance loops. It relates joint velocities to end-effector velocities and maps external Cartesian forces back to joint torques via the relationship $\boldsymbol{\tau} = \mathbf{J}^T\mathbf{F}_{ext}$.

The matrix is derived by computing the partial derivatives of the forward kinematic position equations with respect to the joint variables q_1 and q_2:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \frac{\partial x}{\partial q_2} \\ \frac{\partial y}{\partial q_1} & \frac{\partial y}{\partial q_2} \\ \frac{\partial z}{\partial q_1} & \frac{\partial z}{\partial q_2} \end{bmatrix} = \begin{bmatrix} -L\sin(q_1)\cos(q_2) & -L\cos(q_1)\sin(q_2) \\ L\cos(q_1)\cos(q_2) & -L\sin(q_1)\sin(q_2) \\ 0 & L\cos(q_2) \end{bmatrix}$$

C. Codebase and Simulation

Additional ROS2 code documentation can be found here: **fishing bot code documentation**.

Building out the ROS2 codebase such that the control logic interacts seamlessly with MuJoco simulation and hardware control

TABLE I: Table: Package Descriptions

Package	Description
Bringup	Launch configuration. Can launch simulation or hardware control based on preference.
Description	Mesh and model descriptors.
Control	Tension control package.
Planning	Used to control randomness of 'fish' behavior. Can be thought of as a fish agent (our disturbance).
Sensors	Sensor readings and publishing.
Interfaces	Develop protocol for inter-node communication.

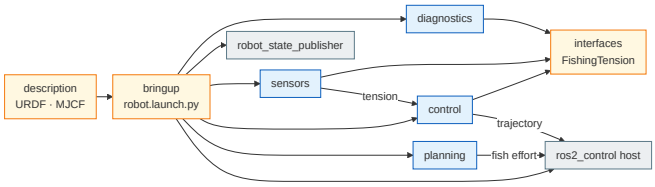


Fig. 3: Package Architecture

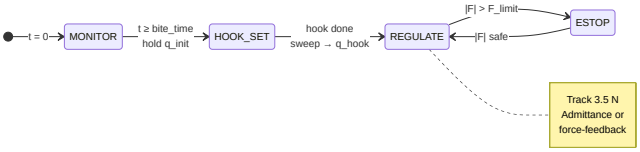


Fig. 4: Fishing State Machine

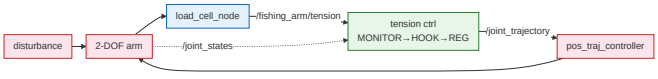


Fig. 5: Control Loop

Simulation Pipeline

The behavior of the fish is simulated as a repeatable disturbance for controlled testing. The controllers were tested in a repeatable way with a ROS2 control structure and MuJoCo simulation.

Fig. 6: Simulation Data Flow

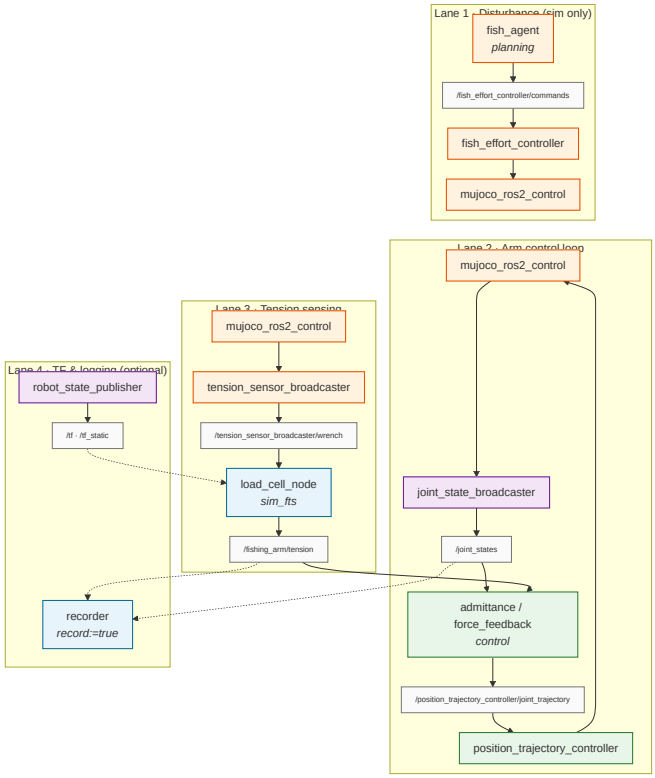


Fig. 7: Software ROS2 graph

Hardware Control Pipeline

The hardware control flow path slightly varies from the simulation pipeline.

Fig. 8: Data Flow

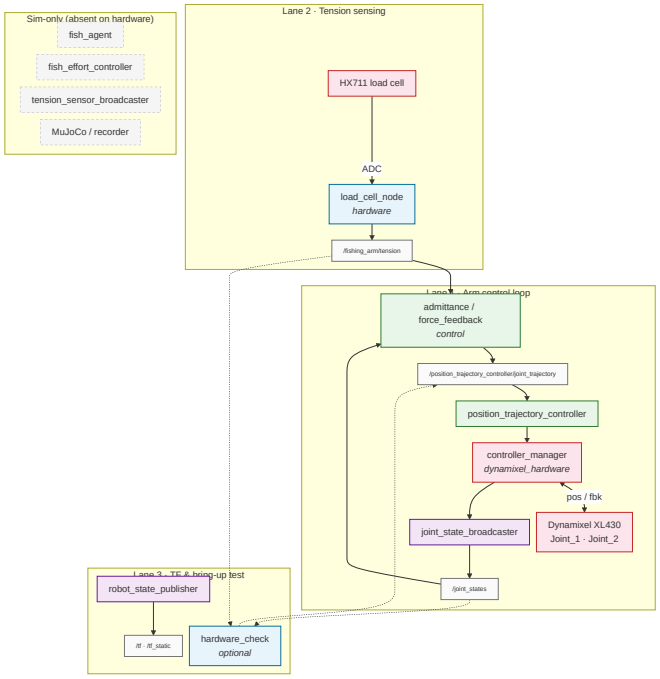


Fig. 9: Hardware ROS2 Graph

D. Control Architecture

Proportional Force Controller

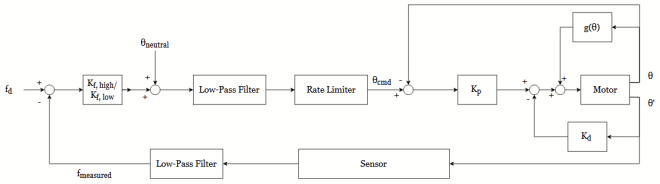


Fig. 10: Proportional Force Control diagram.

The proportional force controller directly regulates the tension of the line by using the tension error measured by the load cell multiplied by an asymmetric gain to brute-force a position command. An inner PD control is then used to track the position of the motor, and a gravity compensation term is injected into the motor command. The system measures the tension error as a difference between the desired and measured tensions

$$e_f = f_d - f_{\text{measured}}$$

An asymmetric, state-dependent gain is then used to map the force error to a corresponding position offset. If the tension error is positive and tension is too high, it uses K_f, high, and if the tension is too low, K_f, low is used. K_f, high < K_f, low . This is such that the rod will 'pull' harder when the tension is low to pick up the slack of the line. Since the fish is hypothetically faster than the motor, the motor has to aggressively compensate for the lack of tension. However, when the tension is too high, the gain is lower to prevent the robot from slamming forwards to yield to the fish, which would cause the line to immediately go slack. The gains are tuned so that there is a balance between the rod being too stiff and unyielding (low K-value) or too soft and pliant, or over-reactive (high K-value).

The inner loop of the controller is a Proportional-Derivative controller that calculates the effort required to move the motor to the commanded position.

$$\tau = K_p(\theta_{cmd} - \theta) - K_d\theta' + g(\theta)$$

In this controller $K_p(\theta_{cmd} - \theta)$ acts as a spring to move the motor towards the command position, K_d dictates how aggressively the controller reacts to the rate of change and acts as a brake, and the gravity compensation term, $g(\theta)$, is used to counteract the downward pull of gravity. Additionally, low-pass filters and rate limiters are used to prevent high spikes in current and to reject extraneous noise or high-frequency actions.

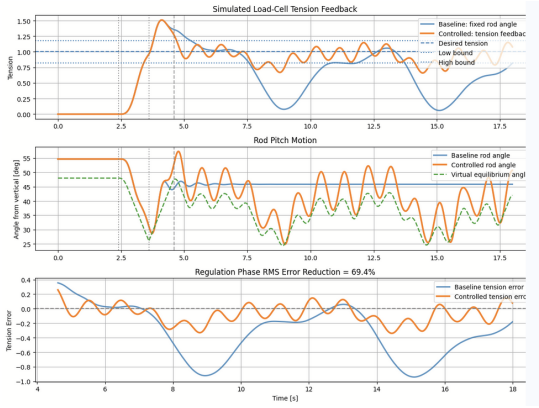


Fig. 11: Performance Comparison Between Fixed-Angle Control and Proportional Force-Based Tension Control

The results of the MujoCo simulation with K_f, high = 160 and K_f, low = 400 are shown in the figure above. The controlled line remains close to the 1.0N desired target and the controller reports a 69.4% RMS Error Reduction. However there is constant chatter in the rod's motion. This is concerning since the motor is constantly reversing directions and the system indicates poor high-frequency rejection or smoothing, even despite the filtering. Because of the system's simplicity and high error reduction, this is the controller chosen for the initial hardware implementation.

Admittance Controller

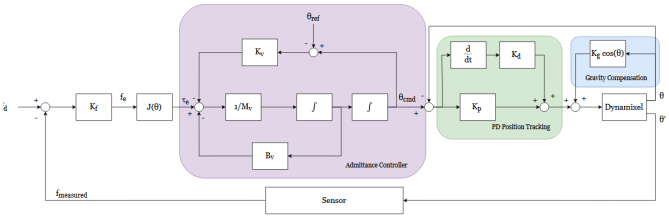


Fig. 12: Admittance Control diagram.

The admittance controller regulates the tension of the line by simulating a virtual mass-spring-damper system, translating measured force errors into a, dynamic position command. An inner PD control is then used to track the simulated position of the motor, and a gravity compensation term is injected into the motor command. The system governs the target angle, θ_{cmd}, by reacting to the tension error, e_f, through the virtual physics equation:

$$M_{v''_{cmd}} + Bv\theta'_{cmd} + K_v(\theta_{cmd} - \theta_{ref}) = -J(\theta)k_f e_f$$

Instead of relying on state-dependent asymmetric gains to manage slack, this architecture relies on sculpting these "virtual physics." The force sensitivity gain, K_f, scales the measured tension error and converts it into a virtual torque using the Jacobian, $J(\theta)$. This torque acts upon a virtual mass, M_v, which dictates the inertial resistance of the rod and naturally filters out high-frequency sensor noise. To prevent the rod from oscillating wildly or yielding too quickly, virtual damping, B_v, acts as a physical shock absorber. Finally, a virtual stiffness, K_v, acts as a backbone, providing a restoring force that pulls the rod back to its neutral fighting angle, θ_{ref}, to prevent the pitch from drifting when the line goes slack. These parameters are tuned to balance a fast, responsive yield against dangerous phase lag or a slow, overly-stiff system. The inner loop of the controller is a Proportional-Derivative controller that calculates the effort required to move the physical motor to the simulated command position.

$$\tau = K_p(\theta_{cmd} - \theta) + K_d(d/dt)(\theta_{cmd} - \theta) + K_g cos(\theta)$$

In this controller, $K_p(\theta_{cmd} - \theta)$ acts as a spring to pull the physical motor towards the virtual target, K_d dictates how aggressively the controller damps the tracking error, and the gravity compensation term, $K_g cos(\theta)$, counteracts the downward pull of gravity. Additionally, because the virtual mass and damping mathematically restrict how fast the command signal can accelerate, the system naturally rejects extraneous noise and high-frequency actions without the need for artificial low-pass filters or strict rate limiters.

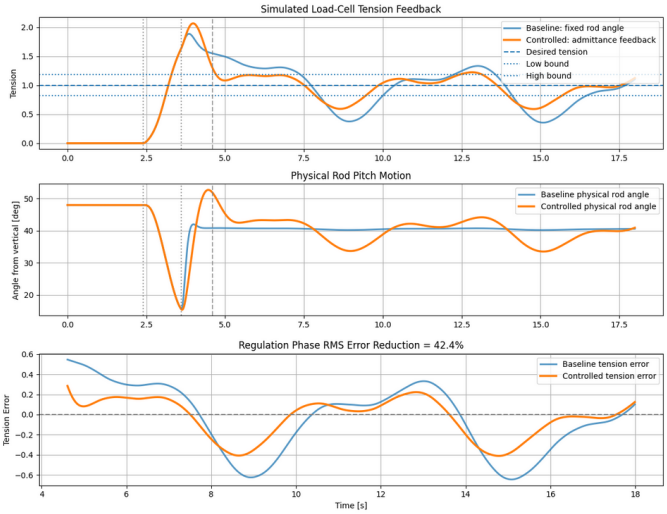


Fig. 13: Performance Comparison Between Fixed-Angle Control and Admittance-Based Tension Control

The results of the MuJoCo simulation are shown in the figure above. The controlled line yields gracefully to the tension spike, and the controller reports a 42.4% RMS Error Reduction. While the numerical error reduction is lower than the proportional controller due to the inherent phase lag required to accelerate the virtual mass, the rod's motion is exceptionally smooth and continuous. This is highly advantageous for hardware longevity, as it completely eliminates the jagged, high-frequency motor chatter and constant direction reversals seen in the proportional approach. However, this controller does require a more complex implementation and tuning process.

IV. Results

A. Error Assessment

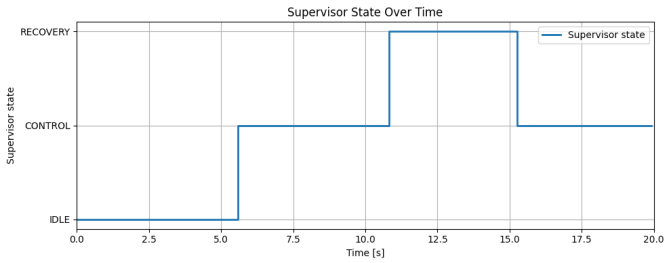


Fig. 14: Engagement and recovery supervisor state during the hardware trial.

1) Engagement and safety logic: The engagement supervisor was also evaluated during the hardware trial. Before the line tension reached the bite-trigger condition, the controller remained in the IDLE state and commanded zero PWM, preventing the motor from responding to sensor noise or small disturbances. In this implementation, the controller was activated only when the measured tension exceeded 0.8 N for 0.10 s, representing the initial pulling event after a fish is hooked. During active control, a low-tension condition was detected when the measured tension stayed below 1.0 N. If this condition lasted for more than 1.8 s, the supervisor entered the RECOVERY state to avoid continuously pulling against a slack line. Once the measured tension recovered above 2.0 N, the controller returned to active control. The supervisor state plot shows this sequence, with transitions from IDLE to CONTROL, then to RECOVERY, and finally back to CONTROL after the tension recovered.

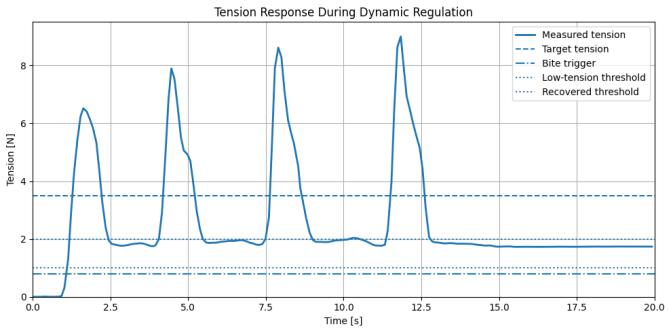


Fig. 15: Measured tension response during dynamic regulation.

2) Dynamic Controller Response: The dynamic controller response was evaluated using a trial with larger variations in the applied line tension. Fig. 8 shows the measured tension signal used as feedback by the controller. Since the line was pulled manually, the sharp peaks in the tension curve should not be interpreted as a standardized disturbance input. The tension signal also depends on factors such as pulling speed, pulling direction, line slack, and the contact condition of the load cell. Therefore, this figure is mainly used to show the feedback signal received by the controller during a more dynamic pulling condition, rather than to evaluate the tension curve alone.

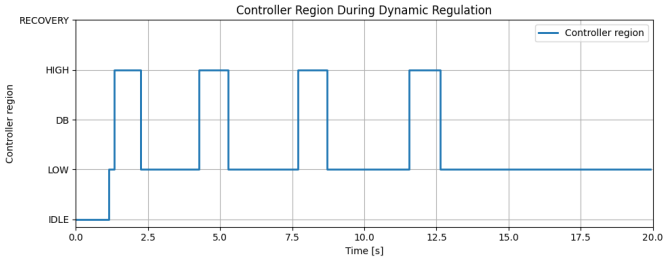


Fig. 16: Controller region switching during dynamic regulation.

The controller response to these tension changes is shown more clearly in Fig. 9 During the trial, the controller compares the measured tension with the target tension and the deadband, then selects the corresponding control region. When the tension rises above the desired range, the controller switches to the HIGH region, which corresponds to a release action to reduce excessive line tension. When the tension drops below the desired range, the controller returns to the LOW region, where the motor pulls back to recover line tension. The repeated LOW/HIGH transitions show that the controller is actively responding to the sign of the tension error rather than applying a fixed motor command.

The LOW and HIGH regions should be interpreted as two operating regions of the same tension-regulation controller, rather than two separate controllers for reeling in and letting out the line. When the measured tension is above the desired range, the controller enters the HIGH region and commands a release action to reduce excessive line tension. When the measured tension is below the desired range, the controller enters the LOW region and commands a pull-back action to rebuild tension. In this sense, the controller is switching its response direction based on the sign of the tension error, while still using one overall feedback-control structure.

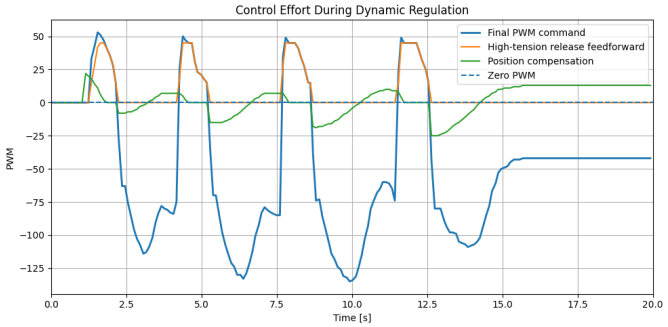


Fig. 17: PWM command and control components during dynamic regulation.

Fig. 10 shows how these feedback decisions are converted into actuator commands. The blue curve is the final PWM command sent to the Dynamixel motor, while the orange and

green curves show two major contributing terms: the high-tension release feedforward and the position compensation term. The zero-PWM line separates the two actuation directions. In this implementation, positive PWM commands are associated with release actions during high-tension events, while negative PWM commands are associated with pull-back corrections when the tension becomes too low.

The positive PWM pulses in Fig. 10 appears when the controller enters the HIGH region, showing that the controller is actively trying to release the rod and reduce excessive tension. After the tension decreases, the final PWM command becomes negative, which corresponds to the LOW-region pull-back action. The position compensation term changes more slowly because it depends mainly on the rod position rather than the instantaneous tension peak. As a result, the final PWM command is the combined output of tension-based correction and position-based compensation.

The PWM command in this hardware trial appears relatively step-like because the prototype was implemented using PWM mode on the Dynamixel motor, rather than a calibrated torque or current-control mode. In addition, the controller uses a deadband-based switching logic, so the control effort changes more noticeably when the measured tension moves between the LOW, deadband, and HIGH regions. Therefore, the PWM signal should not be interpreted as an ideal continuous torque command. Instead, it is used here to show the direction and relative strength of the motor response produced by the tension-regulation logic.

The asymmetry between the positive and negative PWM commands is also intentional. During release, the goal is to reduce excessive tension without moving the rod forward too aggressively. A very aggressive release could either drive the rod close to the forward limit or reduce the line tension too quickly and create slack. For this reason, the release command was kept relatively conservative. During pull-back, however, the motor needs more authority to rebuild line tension while overcoming gravity, friction, and mechanical resistance in the hardware setup. This explains why the negative pull-back commands are larger in magnitude than the positive release commands.

Overall, the agreement between Fig. 8, Fig. 9, and Fig. 10 shows the intended closed-loop behavior of the controller. Fig. 8 provides the measured tension feedback, Fig. 9 shows the corresponding HIGH/LOW region selection based on the tension error, and Fig. 10 shows the motor command generated from this feedback logic. Together, these results indicate that the robot actively responds to changes in line tension by switching control modes and commanding the actuator in the expected direction.

For an actual fishing application, large force oscillations would not be desirable because they could increase the risk of excessive line loading or temporary loss of line tension. In this project, the rapid-pull trial was mainly used as a stress test to check whether the controller could detect sudden tension changes and respond in the correct direction. The results show the expected switching behavior, but they also suggest that smoother mode transitions and improved inner-loop actuation would be useful for reducing oscillations in future versions of the system.

3) Post-Transient Regulation Performance: The post-transient regulation trial was used to evaluate the controller under a more sustained pulling condition after the initial engagement. In this test, an initial manual pull was applied to create a large tension deviation, and the line was then held as steadily as possible. Although the input was still applied by hand and should not be treated as a standardized step input, this trial better represents the controller behavior after the large initial disturbance has passed. It was therefore used to evaluate whether the system could bring the measured tension back near the 3.5 N target and maintain it in the later part of the trial.

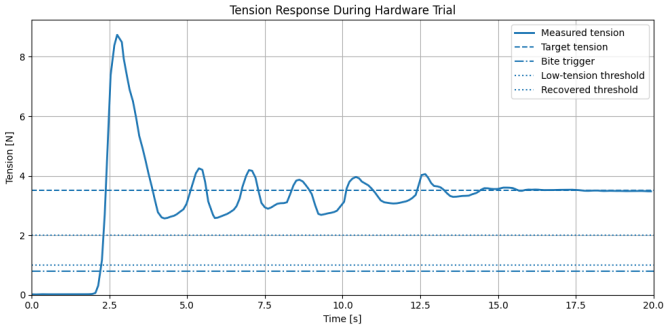


Fig. 18: Measured tension response after the initial engagement transient.

Fig. 11 shows the measured tension response during this trial. The target tension was set to 3.5 N. During the initial engagement phase, the measured tension contains a large transient peak, which is mainly caused by the initial line engagement and manual pulling input. After this transient and the following correction motions, the controller brings the measured tension back toward the target range. In the later part of the trial, the tension remains close to the desired value, showing that the controller can maintain the line tension near the target under a relatively stable pulling condition.

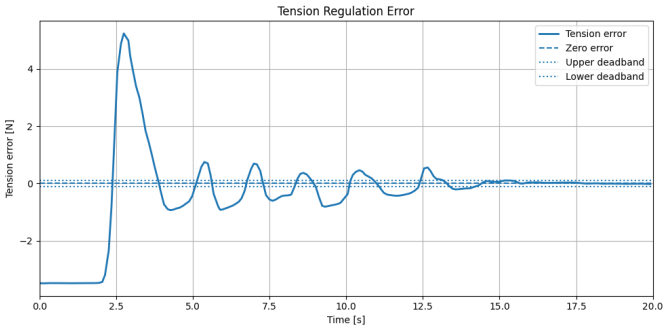


Fig. 19: Tension regulation error after the initial transient.

The tension regulation error is shown in Fig. 12. Instead of treating the full response as a standard settling-time test, the final portion of the trial was used as a post-transient

evaluation window. Over this window, the measured tension had an average value of 3.52 N compared with the 3.5 N target. The mean absolute tension error was 0.025 N, and the RMSE was 0.036 N, corresponding to approximately 0.72% and 1.03% of the target tension, respectively. These values show that once the large transient response had passed, the controller was able to regulate the line tension with a small remaining error.

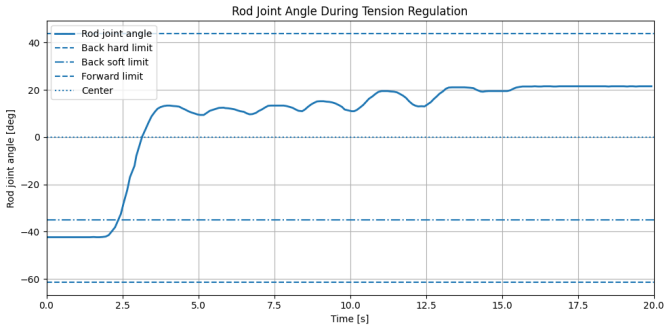


Fig. 20: Rod joint angle during post-transient tension regulation.

Fig. 13 shows the rod joint angle during the same trial, reported in degrees. The joint angle provides additional context for the tension regulation behavior shown in Fig. 11 and Fig. 12. During the initial engagement and correction phase, the tension error is relatively large, and the actuator changes the rod angle to establish line tension and reduce the error. As the measured tension approaches the target in the later part of the trial, the tension error becomes smaller and the rod joint angle also remains near a stable operating posture. This trend indicates that the actuator adjustment is consistent with the reduction of the tension error. However, the joint angle is not expected to match the tension error exactly, since the measured tension also depends on the manual pulling input, line slack, load-cell contact, and friction. Therefore, Fig. 3.3 is used to show the actuator motion and final rod posture, while Fig. 12 remains the main performance metric for tension regulation.

V. Current Challenges

Although the hardware test shows that the controller can regulate the line tension, the current setup still has several practical limitations. One major limitation is that the external line tension was applied by hand during the experiment rather than generated by a standardized disturbance source. This made the test closer to an actual fishing interaction, but it also means that the measured tension curve is affected by the pulling speed, pulling direction, and line slack. As a result, some sharp peaks and sudden drops in the tension signal should be interpreted as part of the manual interaction with the line, instead of being attributed to the controller alone. Therefore, the hardware results are mainly used to demonstrate the controller behavior under realistic manual interaction rather than to provide a standardized disturbance-response benchmark.

Another important limitation comes from the indirect relationship between motor motion and measured line tension. Unlike a pure position-control task, where the motor angle can be measured and controlled directly, the line tension depends on several physical factors between the actuator and the sensor. For example, line slack, fishing-line compliance, load-cell contact condition, and friction in the mechanism can all affect the measured tension even when the motor command is the same. As a result, small motor motions do not always produce an immediate or proportional change in tension.

This limitation is also related to the PWM-based actuation used in the current implementation. Since the motor is commanded through PWM rather than a precise torque or position controller, small corrective commands may need to overcome static friction, gravity, and mechanical resistance before visible rod motion occurs. This can make the actuator response less smooth during fine tension corrections. Therefore, the tension regulation result should be understood as the combined behavior of the controller, motor actuation, fishing line, and load-cell measurement, rather than as a direct one-to-one mapping between joint angle and tension.

VI. Future Work

Upon further understanding of the problem, it became clear that this apparently simple tension management task has incredibly deep and rich control ideas embedded beneath the surface. This led to a deep dive of potential control tools that may bolster the robustness of the control system.

A. Algorithms

First let's consider extensive ideas on the algorithmic front. For one, if we consider taking the 1DOF control problem to the more realistic 2DOF planarized case, nonlinear coupling dynamics and orthogonal subspaces appear. But less obvious is added complication regarding sensing capabilities. Considering our limited 1D load sensing framework, we will require sensor fusion/interpretation techniques to ensure optimal tension estimation in the higher dimensional space (2D). In other words, instead of a direct linear force/tension estimation relation for the 1DOF case, we need to be aware of a more precise tension vector to perform control feedback on. If we use load sensor info and kinematic state via encoders we may employ kinematic variation/dithering to back out the tension vector estimation. But if we decide to use joint torques as well, a sensor fusion technique such as an extended kalman filter may be necessary. Furthermore, the concept of hybrid control may come into play to allow admittance in the tension management force adaptive axis and rigid positional control in perpendicular axes.

Another crucial facet of the control problem is state switching capability. Discrete states are naturally embedded within the fishing control problem, and makes the problem incredibly control rich. The transition from fish searching to hook set, or switching from having fish to losing tension are both examples of discrete switching behavior. The problem can be summarized as this: how do we ensure stable and robust discrete state

switching capability? A robust Finite State Machine (FSM) must be prescribed. The current FSM implementation uses basic force thresholding that induces highly volatile "chatter" behavior, or rapid, unstable, and unnecessary state switching behavior. A few stability insurance techniques come to mind to augment our FSM. We can first employ common FSM techniques, two of which being hysteresis and debouncing. Hysteresis may help better characterize switching conditions by enforcing anisotropic force thresholds for entering and exiting a state. Debouncing adds time-based thresholding/conditioning to limit false/rapid triggering.

Another region of focus in the FSM is the transition points themself. We could apply smoothing filters to help initiate bumpless transfer and eliminate unwanted transient spikes in control effort. A crucial function of the controller is to also place the system under regions of controllability. In this particular problem, we want to avoid fish line snapping or losing tension control on the fishing being hooked onto the line. We can use tension break detection (rate of force change) and accompanying funnel synthesis to bound the tracking error control behavior. The idea of funnel synthesis can be applied to the immediate high error case where tension is suddenly lost. Funnel synthesis drops the robot controller into a safe, heavily damped recovery 'funnel' in case of line break preventing the robotic arm from violently whipping backward and breaking motors. This funnel synthesis concept can also be applied for control regain.

Additionally, the state behaviors themselves have to be tended to. If admittance control is being used we can use gain scheduling to customize dynamic behaviors specific to a state's concerns.

AI has relevance to control enhancement here as well. We can augment the control loop with an adaptive neural network feedforward term that learns the stochastic frequency of the fish's pulling behavior, effectively predicting the required q_d before the tension error grows too large. Our preexisting sim-real frameworks can also be set up to build a Sim-to-real training pipeline. We use our simulation framework as an AI training tool to optimize fish tensioning behavior in the real robot. This can be primarily achieved in two ways: either we use direct RL methods to build a neural-based controller, or employ layered RL to tune controller parameters of a pre-designed control framework. Our simulation pipeline can also simply be used for adaptive control tuning without AI.

B. Noise reduction and filtering techniques

To properly target fishes for catching in an unpredictable open ocean scenario, we may require probabilistic state estimation (HMM, Bayesian filtering). This concept is often important in partial observability/limited info scenarios such as real world RL training cases. Frequency and derivative of the force may be used to distinguish between fake disturbances (boat rocking, snag on seaweed) or an actual fish. However, in the case the environment we are operating in (in our scenario it would be the ocean or a body of water) has no partial observability or obstruction concerns, we may simply bypass the problem state estimation seeks to solve by using computer vision to distinguish between a fish and ignorable entities.

C. Hardware

A few hardware-specific augmentations could be worth studying in the future, to observe their impact on tension management characteristics (transient behaviors, error attenuation, control effort).

The introduction of a redundantly actuated serial joint is one such augmentation with significant control implications. The idea was initially inspired by a desire to emulate rod compliance with active control capability. Then, the larger control picture became clear. The most practical use of the redundant serial joint is the implementation of a macro-micro manipulator structure. The lower heavy base joint handles low-frequency, high-amplitude positioning (like tracking boat's movement). A smaller, redundantly actuated serial joint near the tip can be tuned with a much higher control loop frequency to handle high-frequency stochastic disturbances. As a result, we receive the benefit of inertial decoupling: we get improved torque efficiency as jerky movements are handled by the actuator near the tip and doesn't cause the heavy base motor downstream to respond unnecessarily and move high inertial loads.

The added joint also opens the possibility of null space optimization with the introduction of infinite joint configurations. This allows us to initiate secondary optimization objectives besides tension management such as torque minimization.

The secondary joint can also be treated as a dynamic impedance shaping element, with the redundant motor acting as a dynamically tunable physical stiffness unit that adheres to control needs.

Passive structures are another potential angle for improving dynamic performance. A compliant segment added to the end of the rod can mechanical LPF, dealing with incredibly jerky fish motion in true fishing scenarios as well as improve bumpless transfer stabilization during state transitions to reduce risky torque spikes.

The last hardware-based project enhancement has less to do with the robotic arm itself, but more so about creating an improved real-world testing rig. The idea is to create an actuatable object to mimic a fish that has planarized movement capabilities to perform varied experiments on our hardware system and its accompanying controller design

D. Practical additions

Other considerations that would bring the automated fishing system closer to a 'market-available' product include an actuated hooking system and a tensioning reel mechanism. This would come with its own host of hardware implementation and control framework problems.

ACKNOWLEDGMENT

The author thanks Professor Veronica Santos for guidance and instruction throughout the MAE 263C Controls of Robotics Systems course. Their insights into soft robotic modeling formed the foundation of this project.

References

[1] M. AlYaseen, O. Alajmi, N. Al-Attal, and A. Al-Saleh, "Autonomous Fishing Rod," College of Engineering and Applied Sciences, American University of Kuwait, Kuwait, 2019.

[2] J. Cook, "Robotic Fishing Pole Casts and Reels Automatically," Hackster.io, 2019. [Online]. Available: Hackster.io.

[3] Stanford CHARM Lab, "Haptic Fishing Robot," Stanford University, 2024. [Online]. Available: Stanford CHARM Lab.

[4] Lai, G., Zhou, S., Yang, W., Wang, X., & Wang, F. (2023). Prescribed fixed-time adaptive neural control for manipulators with uncertain dynamics and actuator failures. Mathematics, 11(13), 2925. https://doi.org/10.3390/math11132925 Cited by: 4

VII. Appendix

A. Itemized Breakdown

TABLE II: Itemized breakdown of team member contributions.

Task	Nathan	Julie	Chaoyi	Xiang
CAD / Mechanical Design	90%	10%	0%	0%
ROS2 / Software Architecture	100%	0%	0%	0%
Fabrication / 3D Printing	0%	45%	45%	10%
Sensor / Wiring / Debugging	10%	10%	40%	40%
Controller Design	10%	40%	40%	10%
Simulation / Controller Testing	0%	50%	0%	50%
Hardware Controller Testing	10%	10%	15%	65%
Results / Plotting	10%	10%	10%	70%
Report / Presentation	25%	25%	25%	25%

B. Bill of Materials

TABLE III: Bill of Materials

Item	Component	Qty	Description
1	Arduino Uno R3	1	Sensor reading and hardware communication.
2	HX711	1	Load-cell signal conditioning module.
3	Load Cell	1	Measures fishing-line tension.
4	Dynamixel MX-28R	2	Smart servo motors for rod actuation.
5	U2D2	1	USB-to-Dynamixel communication adapter.
6	12 V DC Power Supply	1	Powers the Dynamixel motors.
7	USB Cable (A–B)	1	Arduino-to-computer connection.
8	Dynamixel TTL Cable	2	Daisy-chain communication between servos.
9	Jumper Wire Set	1 set	Connections between Arduino, HX711, and load cell.
10	Fishing Rod Assembly	1	Custom 1-DOF rod structure.
11	M2 Screws	10	Connect motor to 3D-printed assembly.

C. Custom code

The custom project code is submitted as separate PDF files as part of the Turnitin submission. Only code written specifically for this course project is included. External libraries and built-in package source code, such as MuJoCo, Dynamixel SDK, ROS2, NumPy, and Matplotlib libraries, are not included.

The submitted custom code consists of the following files:

- `MuJoCo_controller.pdf`: custom MuJoCo simulation code for the 1-DOF robotic fishing-rod tension regulation system. This code defines the simulated rod model, fish-like external disturbance, load-cell-style tension feedback, controller states, controller update logic, and visualization/analysis routines used to evaluate the controller response in simulation.
- `project_w_t_controller.pdf`: custom hardware controller code for the final Dynamixel-based tension regulation experiment. This code reads load-cell tension feedback through the Arduino/HX711 interface, filters the measured tension signal, applies the engagement supervisor logic, switches between LOW, deadband, and HIGH tension-control regions, computes the PWM command with tension-based correction and position compensation, sends commands to the Dynamixel motor through the U2D2 interface, and logs experimental data for result plotting.

These code files correspond to the simulation and hardware implementation used in the final report. The MuJoCo code was used for controller testing before hardware deployment, while the hardware controller code was used to collect the experimental results shown in the Results section.

D. Custom CAD designs

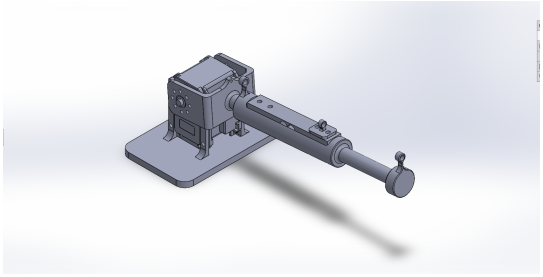


Fig. 21: CAD of robot.

E. Custom systems-level wiring diagrams

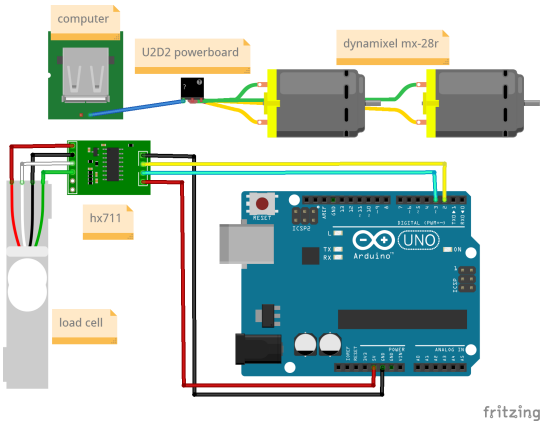


Fig. 22: Wiring diagram.